

MANUAL TÉCNICO

SISTEMA CALIFICACION DE CURSOS

Integrantes del proyecto

Nataly Icu - 202300393

Kelly Ramirez - 20240198

Introduccion

El Sistema de Calificación de Cursos es una aplicación web desarrollada para permitir a los estudiantes consultar información relacionada con los cursos y catedráticos de la carrera de Ciencias y Sistemas.

El sistema permite a los usuarios registrarse e iniciar sesión, administrar sus cursos aprobados, consultar perfiles académicos, crear publicaciones relacionadas con cursos o catedráticos y realizar comentarios sobre dichas publicaciones.

La aplicación utiliza una arquitectura cliente-servidor, donde el frontend se comunica con un servidor mediante una API REST

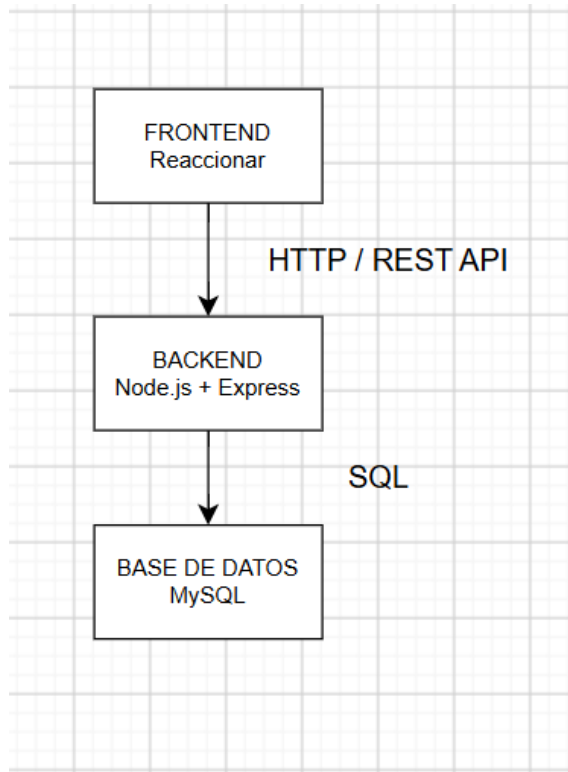
Objetivos

Desarrollar un sistema de calificación que permita gestionar de manera eficiente, segura y organizada la información de los perfiles de los estudiantes, cursos, catedráticos, publicaciones y comentarios sobre los cursos.

- Diseñar e implementar una estructura de backend que permita administrar estudiantes, docentes, cursos, evaluaciones y calificaciones.
- Desarrollar una API REST que permita realizar operaciones de registro, consulta, actualización y eliminación de información académica.
- Documentar la arquitectura del backend, incluyendo la estructura de clases, base de datos y endpoints disponibles de la API REST.

Arquitectura del sistema

Arquitectura cliente-servidor



El sistema utiliza una arquitectura cliente-servidor compuesta por tres componentes principales: frontend, backend y base de datos.

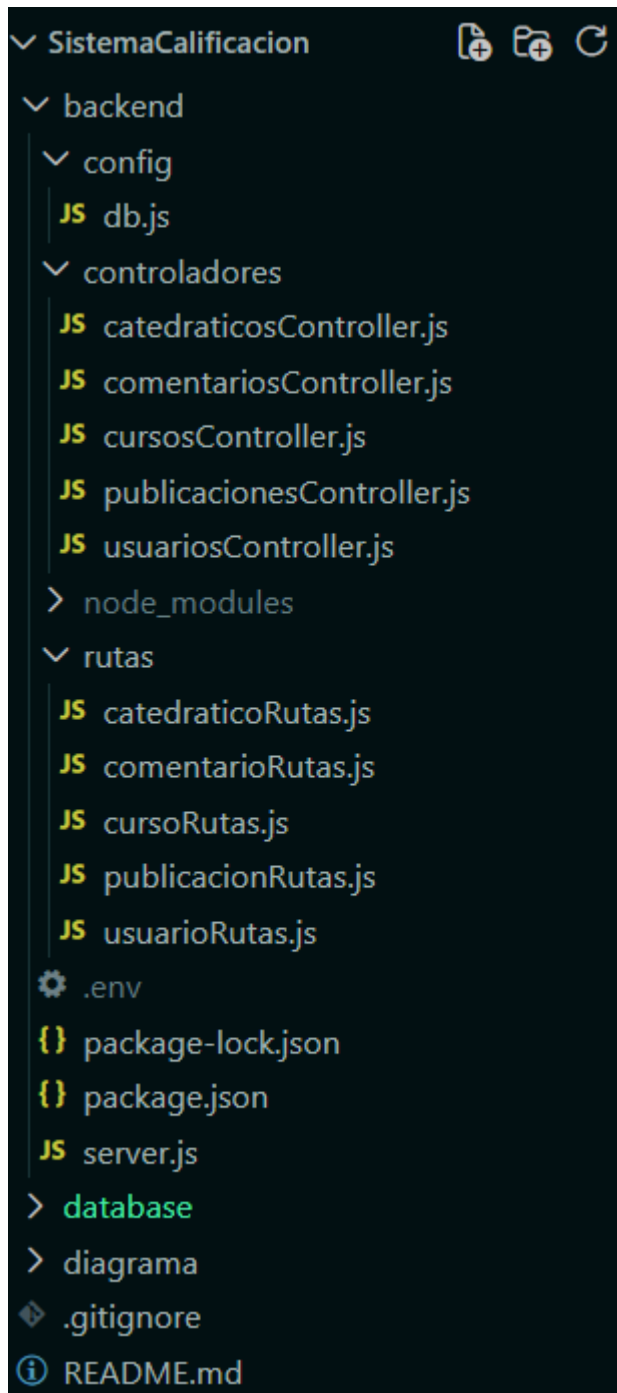
El frontend será desarrollado utilizando React y será responsable de proporcionar la interfaz con la que interactúa el usuario.

El backend está desarrollado con Node.js y Express y proporciona una API REST encargada de recibir las solicitudes del frontend, procesar la información y comunicarse con la base de datos.

La base de datos utiliza MySQL para almacenar la información de usuarios, cursos, catedráticos, publicaciones, comentarios y cursos aprobados.

La comunicación entre el frontend y el backend se realiza mediante solicitudes HTTP utilizando los diferentes métodos de la API REST, mientras que el backend utiliza consultas SQL para interactuar con MySQL.

Arquitectura del servidor



La carpeta config contiene la configuración de la conexión con la base de datos MySQL.

El servidor está organizado utilizando una separación entre rutas, controladores y configuración de la base de datos.

La carpeta controladores contiene la lógica encargada de procesar las solicitudes recibidas, realizar las operaciones correspondientes y generar las respuestas.

El archivo server.js es el punto de entrada de la aplicación. En él se inicializa Express, se configuran los middleware necesarios y se registran las diferentes rutas de la API.

Base de datos y diagrama entidad-relacion

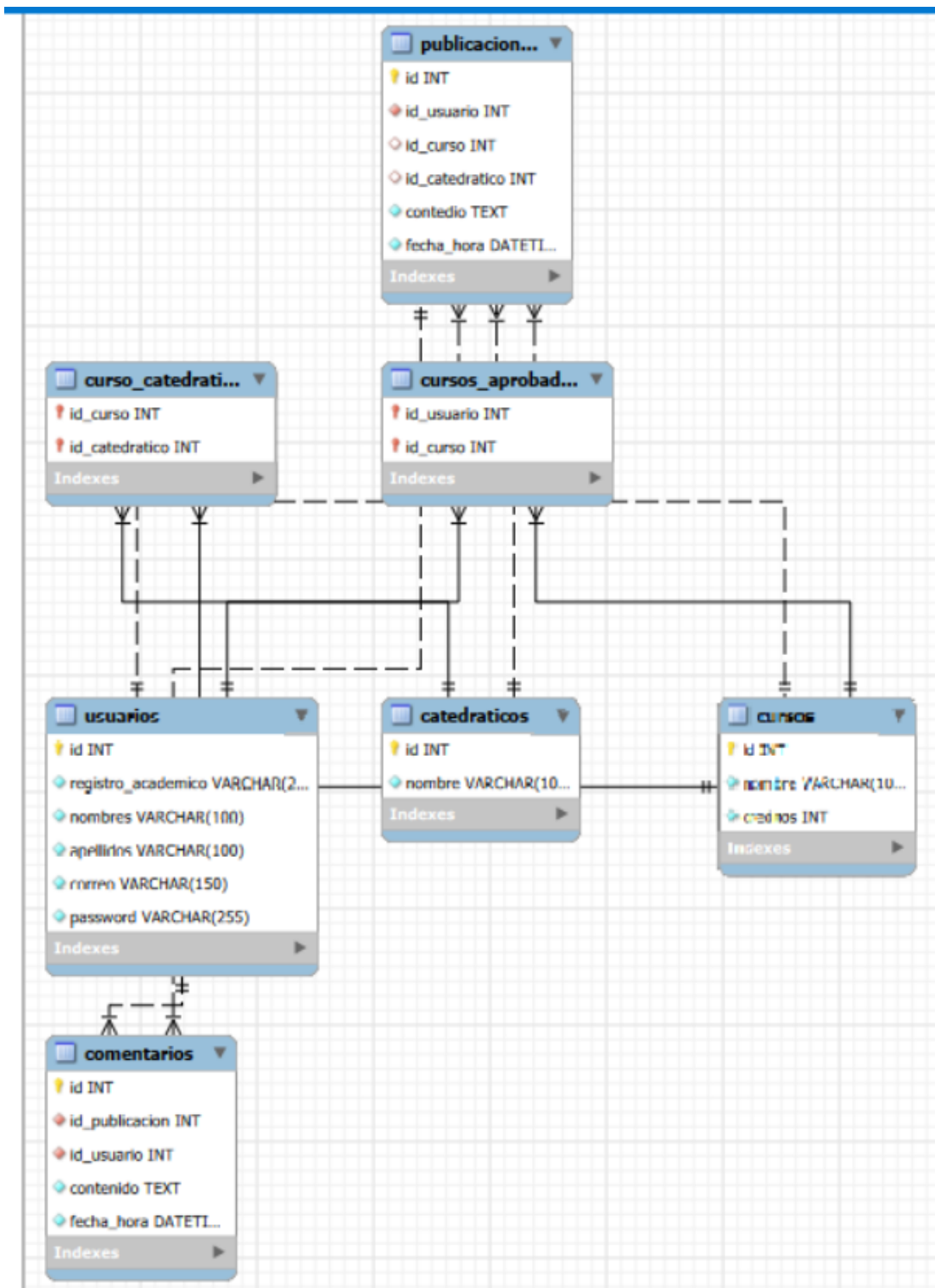
tabla	funciones
USUARIOS	Almacena la informacion de los usuarios
CURSOS	Almacena los cursos disponibles
CATEDRATICOS	Almacena los catedraticos
CURSO_CATEDRATICO	Relaciona cursos con catedraticos
PUBLICACIONES	Almacena las publicaciones
COMENTARIOS	Almacena comentarios
CURSOS_APROBADOS	Relaciona usuarios con cursos aprobados

La base de datos del sistema se implementó utilizando MySQL y está compuesta por siete tablas principales: usuarios, cursos, catedraticos, curso_catedratigo, publicaciones, comentarios y cursos_aprobados.

La tabla usuarios almacena la información de los estudiantes registrados. La tabla cursos contiene los cursos disponibles y sus respectivos créditos. La tabla catedraticos almacena la información de los profesores.

Las tablas curso_catedratigo y cursos_aprobados funcionan como tablas de relación. La primera relaciona los cursos con los catedráticos que los imparten y la segunda relaciona a los usuarios con los cursos que han aprobado.

La tabla publicaciones almacena las publicaciones realizadas por los usuarios y permite asociarlas con un curso o un catedrático. La tabla comentarios almacena los comentarios realizados sobre las publicaciones.



El diagrama Entidad-Relación representa las entidades, atributos, claves primarias, claves foráneas y relaciones existentes entre las tablas.

API REST

metodo	Endpoint	Descripcion
POST	/api/usuarios/registro	registrar usuario
POST	/api/usuarios/login	iniciar sesion
POST	/api/usuarios/recuperar	recuperar contraseña
GET	/api/usuarios/:id	obtener usuario
PUT	/api/usuarios/:id	actualizar usuario
GET	/api/usuarios/buscar/:registro:academico	buscar usuario
GET	/api/usuarios/:id/cursos-aprobados	obtener cursos aprobados
POST	/api/usuarios/:id/cursos-aprobados	agregar curso aprobado
DELETE	/api/usuarios/:id/cursos-aprobados/:id_curso	/eliminar curso aprobado
GET	/api/usuarios/:id/total-creditos	obtener total de creditos
GET	/api/usuarios/perfil/:registro_academico	obtener perfil completo
GET	/api/cursos	obtener cursos
GET	/api/catedraticos	obtener catedraticos
GET	/api/catedraticos/:id/cursos	obtener cursos de un catedratico
POST	/api/publicaciones	crear publicacion
GET	/api/publicaciones	obtener publicaciones
POST	/api/comentarios	crear comentario
GET	/api/comentarios/publicaciones/:id_publicacion	obtener comentario

Especificaciones de endpoint

POST /api/usuarios/registro

Permite registrar un nuevo usuario en el sistema.

Body:

```
{  
  "registro_academico": "123456",  
  "nombres": "Nombre",  
  "apellidos": "Apellido",  
  "correo": "correo@example.com",  
  "password": "123456"  
}
```

Respuesta exitosa:

```
{  
  "mensaje": "Usuario registrado correctamente"  
}
```

POST /api/usuarios/login

Permite iniciar sesión utilizando el registro académico y la contraseña.

Body:

```
{  
  "registro_academico": "123456",  
  "password": "123456"  
}
```

GET /api/cursos

Obtiene todos los cursos registrados en el sistema.

Respuesta: Lista de cursos con su identificador, nombre y cantidad de créditos.

GET /api/catedraticos

Obtiene la lista de catedráticos registrados.

GET /api/catedraticos//cursos

Obtiene los cursos asociados a un catedrático específico.

Parámetro: id – identificador del catedrático.

POST /api/publicaciones

Permite crear una publicación relacionada con un curso o catedrático.

Body:

```
{  
  "id_usuario": 1,  
  "id_curso": 7,  
  "contenido": "El curso me parece interesante."  
}
```

POST /api/comentarios

Permite agregar un comentario a una publicación.

Body:

```
{  
  "id_publicacion": 1,  
  "id_usuario": 1,  
  "contenido": "Estoy de acuerdo."  
}
```

GET /api/comentarios/publicacion/

Obtiene los comentarios pertenecientes a una publicación específica.

GET /api/usuarios/perfil/

Obtiene la información completa de un usuario mediante su registro académico, incluyendo sus cursos aprobados y el total de créditos

PRUEBAS Y EJECUCION

Para comprobar el funcionamiento de la API REST se utilizó Thunder Client, realizando solicitudes HTTP a los diferentes endpoints del servidor.

Se realizaron pruebas para verificar el registro e inicio de sesión de usuarios, consulta de cursos y catedráticos, creación y consulta de publicaciones, aplicación de filtros, creación y consulta de comentarios y consulta de perfiles académicos.

El servidor se ejecuta mediante Node.js utilizando el comando:

```
node server.js
```

El servidor queda disponible en:

<http://localhost:3000>

servidor ejecutandose correctamente

```
PS C:\Users\marle\Documents\SistemaCalificacion\backend> node server.js
◇ injected env (5) from .env // tip: ~ auth for agents [www.vestauth.com]
Servidor ejecutándose en http://localhost:3000
```

Prueba de publicacion

The screenshot shows the Thunder Client interface with a GET request to `http://localhost:3000/api/publicaciones` successfully executed. The status is 200 OK, the size is 304 Bytes, and the time is 62 ms. The response is a JSON array containing one object representing a publication.

parameter	value
parameter	value

Status: 200 OK Size: 304 Bytes Time: 62 ms

```
1  [
2    {
3      "id": 1,
4      "contenido": "El curso de Matemática Básica 1 me parece bastante interesante.",
5      "fecha_hora": "2026-09-04T20:50:14.000Z",
6      "id_usuario": 1,
7      "registro_academico": "202600001",
8      "nombres": "kelly".
```

Consulta de publicaciones y filtros

GET ⌵ http://localhost:3000/api/publicaciones?curso=7 Se

Query Headers ² Auth Body Tests Pre Run

Query Parameters

☒	curso	7
☐	parameter	value

Status: 200 OK Size: 304 Bytes Time: 9 ms Response

```
1 [
2   {
3     "id": 1,
4     "contenido": "El curso de Matemática Básica 1 me parece bastante
5     interesante.",
6     "fecha_hora": "2026-09-04T20:50:14.000Z",
7     "id_usuario": 1,
8     "registro_academico": "202600001",
9     "nombres": "Kelly".
```

Response Chart

Prueba de comentarios

POST ⌵ http://localhost:3000/api/comentarios

Query Headers ² Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "id_publicacion": 1,
3   "id_usuario": 1,
4   "contenido": "Estoy de acuerdo, también me parece interesante."
5 }
```

Status: 201 Created Size: 52 Bytes Time: 16 ms Response

```
1 {
2   "mensaje": "Comentario creado correctamente",
3   "id": 2
4 }
```

Consulta de perfil completo

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3000/api/usuarios/buscar/202600001
- Query Parameters:** A table with one row: parameter | value.
- Status:** 200 OK
- Size:** 119 Bytes
- Time:** 6 ms
- Response:** A JSON object containing user profile data.

```
1 {  
2   "id": 1,  
3   "registro_academico": "202600001",  
4   "nombres": "Kelly",  
5   "apellidos": "Ramirez",  
6   "correo": "kellyactualizada@gmail.com"  
7 }
```